

Parte práctica del aporte

Aplicaciones distribuidas

José Alejandro Lozano Morales

7mo Software “A”

Martes 2 de junio del 2026

Repositorio: [Aplicaciones-Distribuidas_7moSemestre/Aporte 02-06](#)



Índice

1. Estructura y archivos usados	1
2. Comunicación entre Cliente y Nodos	2
2.1. Código de ClienteTCP.java	3
2.2. Código de N1.java	5
2.3. Código de N2.java	6
2.4. Código de N3.java	6
2.5. Casos de prueba de comunicación	7
2.5.1. Caso 1: Ejecución de los tres nodos	7
2.5.2. Caso 2: Heartbeats entre nodos	8
2.5.3. Caso 3: Registro correcto de una operación	10
2.5.4. Caso 4: Token inválido	11
2.5.5. Caso 5: Caída de N3 y elección de coordinador	12
2.5.6. Caso 6: Cliente intenta usar un nodo apagado	14
2.5.7. Caso 7: Cliente se recupera usando otro nodo activo	15
3. Algoritmo Bully	16
3.1. Código de NodoInfo.java	16
3.2. Código de Bully.java	18
3.3. Relación con Nodo.java	22
4. Evento	23
4.1. Código de Evento.java	23
5. Lamport	24
5.1. Código de LamportClock.java	24
5.2. Código de Lamport.java	26
5.3. Importancia dentro del prototipo	29

1. Estructura y archivos usados

El prototipo **RegistroDistribuido** está implementado en Java 21 con Maven. Su objetivo es simular un sistema distribuido formado por tres nodos réplica, comunicación TCP, relojes lógicos de Lamport, detección de fallos mediante heartbeats, elección de coordinador con el algoritmo Bully y autenticación mediante token.

La siguiente tabla resume los archivos principales utilizados en el proyecto:

Archivo	Descripción
aportetcp/ClienteTCP.java	Cliente usado para seleccionar un nodo destino y enviar operaciones mediante TCP. Permite probar operaciones válidas, errores de conexión y comportamiento ante nodos apagados.
bully/Bully.java	Archivo relacionado con la lógica del algoritmo Bully. Su propósito es apoyar la elección de un coordinador dentro del sistema distribuido.
evento/Evento.java	Representa un evento registrado en el sistema. Contiene la marca de tiempo Lamport, el ID del nodo y una descripción. También permite ordenar eventos por tiempo lógico.
lamport/Lamport.java	Clase de simulación o demostración del funcionamiento de relojes lógicos de Lamport entre procesos.
lamport/LamportClock.java	Implementa el reloj lógico de Lamport. Permite incrementar el reloj en eventos internos, envíos y recepciones de mensajes.
nodo/N1.java	Lanzador del nodo N1. Configura el nodo con ID 1 y puerto 9001.
nodo/N2.java	Lanzador del nodo N2. Configura el nodo con ID 2 y puerto 9002.
nodo/N3.java	Lanzador del nodo N3. Configura el nodo con ID 3 y puerto 9003.
nodo/Nodo.java	Clase principal del sistema. Integra el servidor TCP, recepción de operaciones, validación de token, heartbeats, detección de fallos, registro de eventos, reloj Lamport y elección de coordinador.

nodo/NodoInfo.java	Representa la información conocida de cada nodo: ID, host, puerto, último heartbeat recibido y estado activo o caído.
pom.xml	Archivo de configuración Maven del proyecto. Define la compilación del proyecto y la versión de Java utilizada.
README.md	Documento con instrucciones para abrir, compilar, ejecutar y probar el prototipo.
ANALISIS.md	Documento de defensa técnica donde se explican decisiones relacionadas con CAP, falacias de la computación distribuida, transparencia, SLA y comparación entre Bully y Raft.

2. Comunicación entre Cliente y Nodos

La comunicación entre el cliente y los nodos se realiza mediante TCP. El programa `ClienteTCP.java` permite seleccionar uno de los tres nodos disponibles: N1, N2 o N3. Cada nodo escucha en un puerto diferente: N1 en el puerto 9001, N2 en el puerto 9002 y N3 en el puerto 9003.

El cliente no mantiene una conexión permanente desde el momento en que se selecciona el nodo. La selección inicial solamente define el destino al que se enviarán las operaciones. La conexión TCP real se intenta cuando el usuario escribe una operación. Esto permite que, si un nodo está apagado, el cliente muestre un error de conexión y recomiende probar con otro nodo activo.

El formato de mensaje utilizado por el cliente es el siguiente:

```
TOKEN123|Operacion
```

Por ejemplo:

```
TOKEN123|Registrar entrada de paquete 001
```

El token `TOKEN123` funciona como un mecanismo simple de autenticación. Si el mensaje no contiene el token esperado, el nodo rechaza la operación. Si el token es correcto, el nodo registra la operación como un evento, actualiza su reloj lógico de Lamport y responde al cliente indicando que la operación fue registrada.

2.1. Código de ClienteTCP.java

```
1 package src.ec.edu.uteq.distribuidas.aportetcp;
2
3 import java.io.*;
4 import java.net.Socket;
5 import java.util.Scanner;
6
7 public class ClienteTCP {
8
9     private static final String TOKEN = "TOKEN123";
10
11     public static void main(String[] args) {
12         Scanner scanner = new Scanner(System.in);
13
14         System.out.println("Seleccione el nodo al que desea
15             conectarse:");
16         System.out.println("1. N1 - puerto 9001");
17         System.out.println("2. N2 - puerto 9002");
18         System.out.println("3. N3 - puerto 9003");
19         System.out.print("Opci n: ");
20
21         int opcion = Integer.parseInt(scanner.nextLine());
22
23         int puerto;
24
25         switch (opcion) {
26             case 1 -> puerto = 9001;
27             case 2 -> puerto = 9002;
28             case 3 -> puerto = 9003;
29             default -> {
30                 System.out.println("Opci n inv lida.");
31                 return;
32             }
33         }
34
35         String host = "localhost";
36
37         System.out.println("Nodo seleccionado: N" + opcion + " en "
38             + host + ": " + puerto);
39         System.out.println("La conexi n TCP se intentar al
40             enviar cada operaci n.");
```

```

38     System.out.println("Formato: TOKEN123|Operacion");
39     System.out.println("Ejemplo: TOKEN123|Registrar entrada de
    paquete 001");
40     System.out.println("Escriba SALIR para cerrar el cliente");
41     System.out.println("
    -----");
42
43     while (true) {
44         System.out.print("> ");
45         String operacion = scanner.nextLine();
46
47         if ("SALIR".equalsIgnoreCase(operacion)) {
48             System.out.println("Cliente finalizado.");
49             break;
50         }
51
52         try (
53             Socket socket = new Socket(host, puerto);
54             PrintWriter salida = new PrintWriter(socket.
                getOutputStream(), true);
55             BufferedReader entrada = new BufferedReader(new
                InputStreamReader(socket.getInputStream()))
56         ) {
57             salida.println(operacion);
58
59             String respuesta = entrada.readLine();
60
61             if (respuesta != null) {
62                 System.out.println("Servidor: " + respuesta);
63             } else {
64                 System.out.println("Servidor sin respuesta.");
65             }
66
67         } catch (IOException e) {
68             System.out.println("No se pudo conectar con el nodo
                en puerto " + puerto);
69             System.out.println("Detalle: " + e.getMessage());
70             System.out.println("Pruebe conectarse a otro nodo
                activo.");
71         }
72     }

```

```

73
74         scanner.close();
75     }
76 }

```

Listing 1: Código completo de ClienteTCP.java

2.2. Código de N1.java

El archivo N1.java funciona como lanzador del primer nodo réplica. Su responsabilidad es crear la información de los demás nodos, configurar a N1 con el ID 1 y el puerto 9001, y luego iniciar el servidor del nodo.

```

1 package src.ec.edu.uteq.distribuidas.nodo;
2
3 import src.ec.edu.uteq.distribuidas.lamport.LamportClock;
4
5 import java.util.ArrayList;
6 import java.util.HashMap;
7
8 public class N1 {
9
10     public static void main(String[] args) {
11         HashMap<Integer, NodoInfo> otrosNodos = new HashMap<>();
12
13         otrosNodos.put(2, new NodoInfo(2, "localhost", 9002));
14         otrosNodos.put(3, new NodoInfo(3, "localhost", 9003));
15
16         Nodo nodo = new Nodo(
17             1,
18             9001,
19             new LamportClock("N1"),
20             otrosNodos,
21             new ArrayList<>()
22         );
23
24         nodo.iniciarServidor();
25     }
26 }

```

Listing 2: Código completo de N1.java

2.3. Código de N2.java

El archivo N2.java funciona como lanzador del segundo nodo réplica. Configura a N2 con el ID 2 y el puerto 9002. También registra la información de los otros nodos para que pueda comunicarse con ellos mediante heartbeats y participar en la elección de coordinador.

```
1 package src.ec.edu.uteq.distribuidas.nodo;
2
3 import src.ec.edu.uteq.distribuidas.lamport.LamportClock;
4
5 import java.util.ArrayList;
6 import java.util.HashMap;
7
8 public class N2 {
9
10     public static void main(String[] args) {
11         HashMap<Integer, NodoInfo> otrosNodos = new HashMap<>();
12
13         otrosNodos.put(1, new NodoInfo(1, "localhost", 9001));
14         otrosNodos.put(3, new NodoInfo(3, "localhost", 9003));
15
16         Nodo nodo = new Nodo(
17             2,
18             9002,
19             new LamportClock("N2"),
20             otrosNodos,
21             new ArrayList<>()
22         );
23
24         nodo.iniciarServidor();
25     }
26 }
```

Listing 3: Código completo de N2.java

2.4. Código de N3.java

El archivo N3.java funciona como lanzador del tercer nodo réplica. Configura a N3 con el ID 3 y el puerto 9003. Cuando los tres nodos están activos, N3 suele ser el coordinador inicial porque posee el ID más alto dentro del algoritmo Bully.

```
1 package src.ec.edu.uteq.distribuidas.nodo;
```



```

2
3 import src.ec.edu.uteq.distribuidas.lamport.LamportClock;
4
5 import java.util.ArrayList;
6 import java.util.HashMap;
7
8 public class N3 {
9
10     public static void main(String[] args) {
11         HashMap<Integer, NodoInfo> otrosNodos = new HashMap<>();
12
13         otrosNodos.put(1, new NodoInfo(1, "localhost", 9001));
14         otrosNodos.put(2, new NodoInfo(2, "localhost", 9002));
15
16         Nodo nodo = new Nodo(
17             3,
18             9003,
19             new LamportClock("N3"),
20             otrosNodos,
21             new ArrayList<>()
22         );
23
24         nodo.iniciarServidor();
25     }
26 }

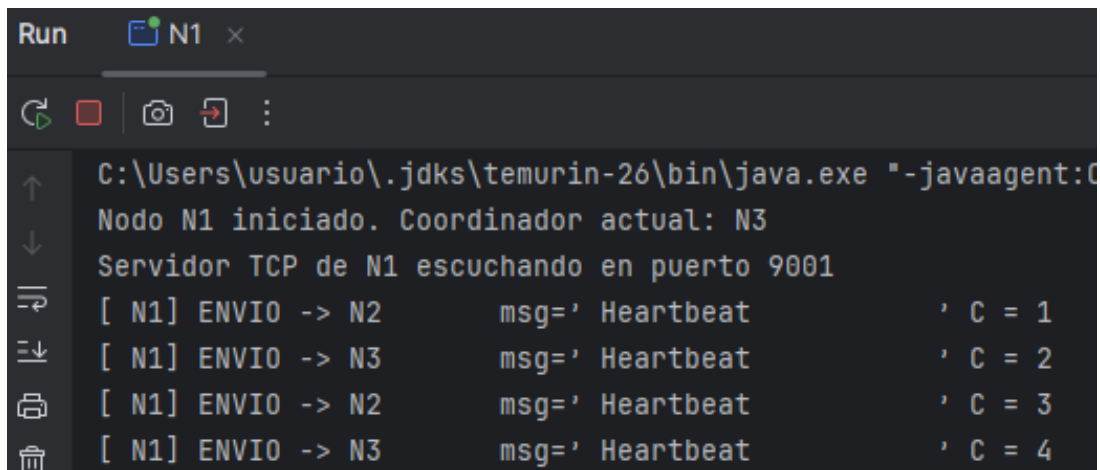
```

Listing 4: Código completo de N3.java

2.5. Casos de prueba de comunicación

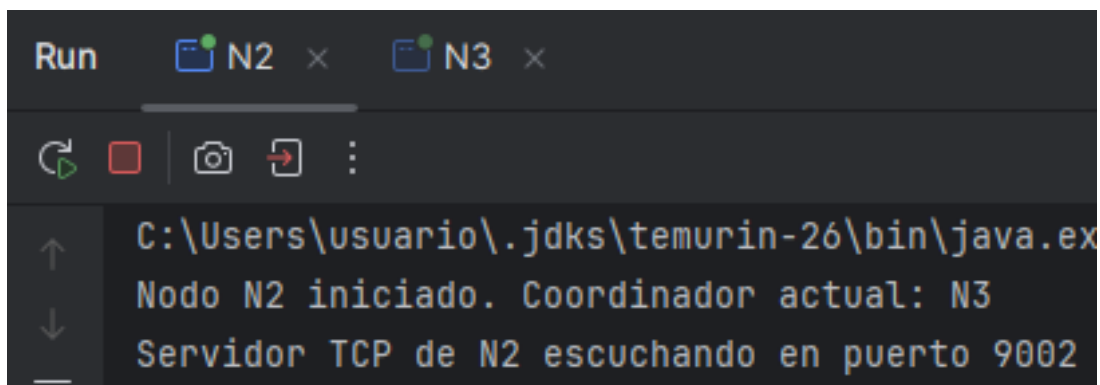
2.5.1. Caso 1: Ejecución de los tres nodos

Este caso demuestra que existen tres réplicas activas dentro del sistema. Se ejecutan las clases N1.java, N2.java y N3.java, cada una escuchando en su puerto correspondiente.



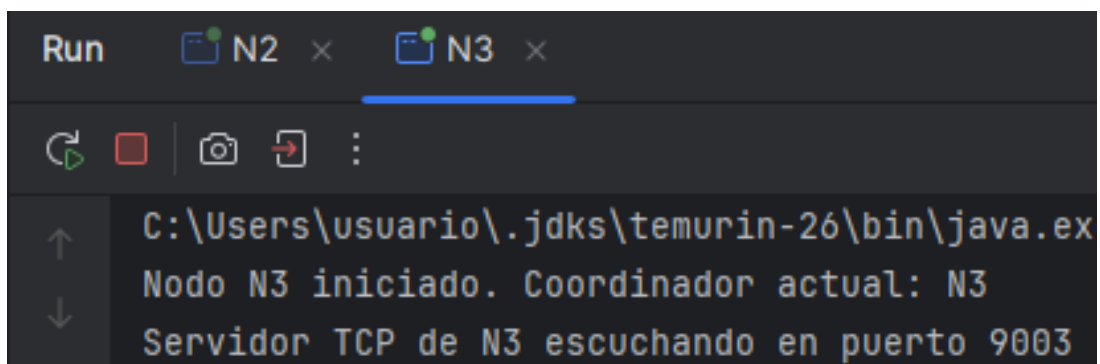
```
Run  N1 x
C:\Users\usuario\.jdk\temurin-26\bin\java.exe -javaagent:0
Nodo N1 iniciado. Coordinador actual: N3
Servidor TCP de N1 escuchando en puerto 9001
[ N1] ENVIO -> N2      msg=' Heartbeat          ' C = 1
[ N1] ENVIO -> N3      msg=' Heartbeat          ' C = 2
[ N1] ENVIO -> N2      msg=' Heartbeat          ' C = 3
[ N1] ENVIO -> N3      msg=' Heartbeat          ' C = 4
```

Figura 1: Ejecución del nodo 1



```
Run  N2 x  N3 x
C:\Users\usuario\.jdk\temurin-26\bin\java.exe
Nodo N2 iniciado. Coordinador actual: N3
Servidor TCP de N2 escuchando en puerto 9002
```

Figura 2: Ejecución del nodo 2



```
Run  N2 x  N3 x
C:\Users\usuario\.jdk\temurin-26\bin\java.exe
Nodo N3 iniciado. Coordinador actual: N3
Servidor TCP de N3 escuchando en puerto 9003
```

Figura 3: Ejecución del nodo 3

2.5.2. Caso 2: Heartbeats entre nodos

Este caso evidencia la comunicación interna entre los nodos. Los nodos envían y reciben heartbeats de forma periódica para comprobar si las demás réplicas continúan activas.

```

Elección Bully en N1: coordinador anterior=N2, nuevo=N3
[ N1] ENVIO -> N2      msg=' Heartbeat      '
[ N1] ENVIO -> N3      msg=' Heartbeat      '
[ N1] RECEPCION <- N2   msg=' Heartbeat recibido'
[ N1] RECEPCION <- N3   msg=' Heartbeat recibido'
[ N1] ENVIO -> N2      msg=' Heartbeat      '
[ N1] ENVIO -> N3      msg=' Heartbeat      '
[ N1] RECEPCION <- N2   msg=' Heartbeat recibido'
[ N1] RECEPCION <- N3   msg=' Heartbeat recibido'
[ N1] ENVIO -> N2      msg=' Heartbeat      '
[ N1] ENVIO -> N3      msg=' Heartbeat      '
[ N1] RECEPCION <- N2   msg=' Heartbeat recibido'
[ N1] RECEPCION <- N3   msg=' Heartbeat recibido'

```

Figura 4: Envío y recepción de heartbeats en nodo 1

```

[ N2] ENVIO -> N1      msg=' Heartbeat      '
[ N2] ENVIO -> N3      msg=' Heartbeat      '
[ N2] RECEPCION <- N3   msg=' Heartbeat recibido'
[ N2] RECEPCION <- N1   msg=' Heartbeat recibido'
[ N2] ENVIO -> N1      msg=' Heartbeat      '
[ N2] ENVIO -> N3      msg=' Heartbeat      '
[ N2] RECEPCION <- N3   msg=' Heartbeat recibido'
[ N2] RECEPCION <- N1   msg=' Heartbeat recibido'
[ N2] ENVIO -> N1      msg=' Heartbeat      '
[ N2] ENVIO -> N3      msg=' Heartbeat      '
[ N2] RECEPCION <- N3   msg=' Heartbeat recibido'
[ N2] RECEPCION <- N1   msg=' Heartbeat recibido'
[ N2] ENVIO -> N1      msg=' Heartbeat      '
[ N2] ENVIO -> N3      msg=' Heartbeat      '

```

Figura 5: Envío y recepción de heartbeats en nodo 2

```

[ N3] ENVIO -> N1      msg=' Heartbeat
[ N3] ENVIO -> N2      msg=' Heartbeat
[ N3] RECEPCION <- N1   msg=' Heartbeat recibido
[ N3] RECEPCION <- N2   msg=' Heartbeat recibido
[ N3] ENVIO -> N1      msg=' Heartbeat
[ N3] ENVIO -> N2      msg=' Heartbeat
[ N3] RECEPCION <- N1   msg=' Heartbeat recibido
[ N3] RECEPCION <- N2   msg=' Heartbeat recibido
[ N3] ENVIO -> N1      msg=' Heartbeat
[ N3] ENVIO -> N2      msg=' Heartbeat
[ N3] RECEPCION <- N1   msg=' Heartbeat recibido
[ N3] RECEPCION <- N2   msg=' Heartbeat recibido
[ N3] ENVIO -> N1      msg=' Heartbeat
[ N3] ENVIO -> N2      msg=' Heartbeat
[ N3] RECEPCION <- N1   msg=' Heartbeat recibido
[ N3] RECEPCION <- N2   msg=' Heartbeat recibido

```

Figura 6: Envío y recepción de heartbeats en nodo 3

2.5.3. Caso 3: Registro correcto de una operación

Este caso demuestra que el cliente puede enviar una operación a un nodo activo usando el token correcto.

TOKEN123|Registrar entrada de paquete 001

```
Run  N1 x N2 x N3 x ClienteTCP x
C:\Users\usuario\.jdk\temurin-26\bin\java.exe "-javaagent:C:\Program Files\
Seleccione el nodo al que desea conectarse:
1. N1 - puerto 9001
2. N2 - puerto 9002
3. N3 - puerto 9003
Opción: 2
Nodo seleccionado: N2 en localhost: 9002
La conexión TCP se intentará al enviar cada operación.
Formato: TOKEN123|Operacion
Ejemplo: TOKEN123|Registrar entrada de paquete 001
Escriba SALIR para cerrar el cliente
-----
> TOKEN123|Registrar entrada de paquete 001
Servidor: OK:N2: operación registrada: Lamport=1423: coordinador=N3
> |
```

Figura 7: Registro de operación desde ClienteTCP

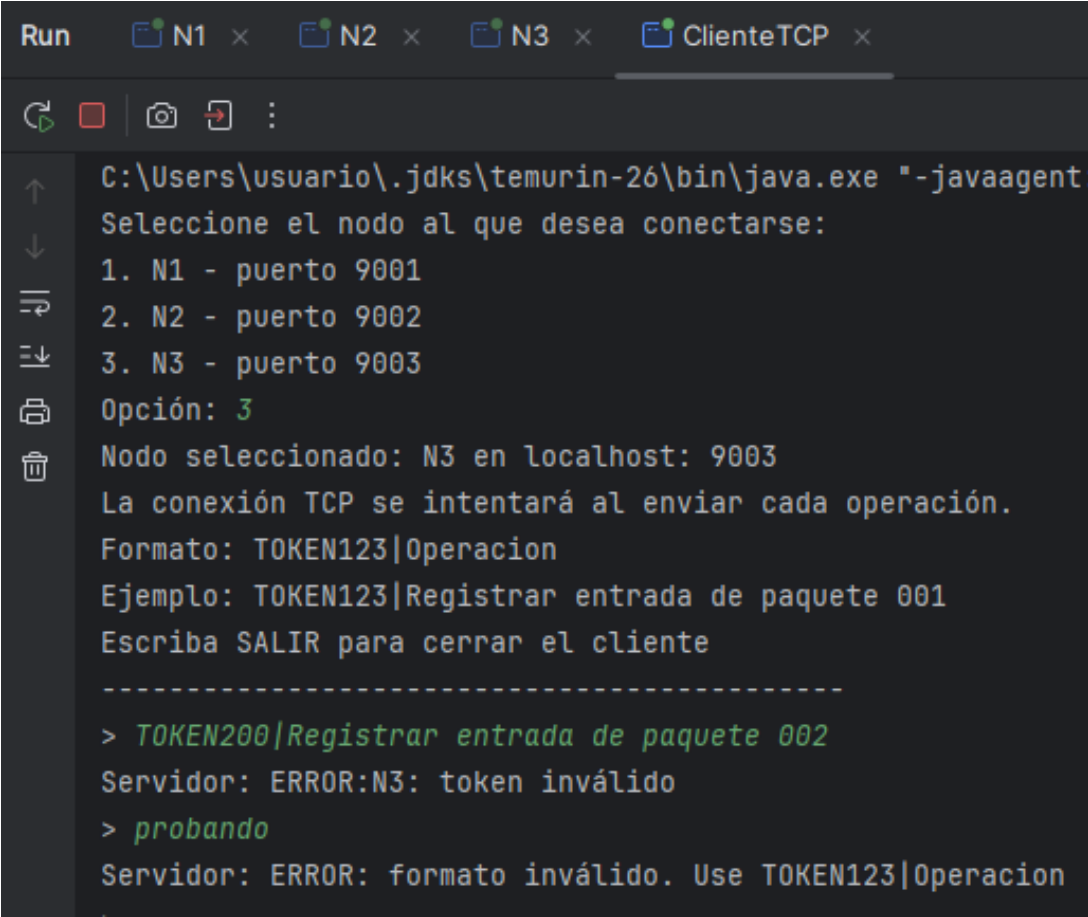
```
Run  N1 x N2 x N3 x ClienteTCP x
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 1406 C= 1407
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido '  marcaMsg= 1407 C= 1408
[ N2] ENVIO -> N1          msg=' Heartbeat              '  C = 1409
[ N2] ENVIO -> N3          msg=' Heartbeat              '  C = 1410
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 1413 C= 1414
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido '  marcaMsg= 1414 C= 1415
[ N2] ENVIO -> N1          msg=' Heartbeat              '  C = 1416
[ N2] ENVIO -> N3          msg=' Heartbeat              '  C = 1417
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 1420 C= 1421
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido '  marcaMsg= 1421 C= 1422
[ N2] EVENTO INTERNO N2 registra operación: Registrar entrada de paquete 001
Log ordenado de N2:
```

Figura 8: Recepción de la operación en nodo 2

2.5.4. Caso 4: Token inválido

Este caso demuestra que el sistema no acepta cualquier mensaje enviado por el cliente. Si el token es incorrecto o el formato no cumple lo esperado, el nodo rechaza la operación.

TOKEN200|Registrar entrada de paquete 002



```
Run N1 x N2 x N3 x ClienteTCP x
C:\Users\usuario\.jdk\temurin-26\bin\java.exe "-javaagent:
Seleccione el nodo al que desea conectarse:
1. N1 - puerto 9001
2. N2 - puerto 9002
3. N3 - puerto 9003
Opción: 3
Nodo seleccionado: N3 en localhost: 9003
La conexión TCP se intentará al enviar cada operación.
Formato: TOKEN123|Operacion
Ejemplo: TOKEN123|Registrar entrada de paquete 001
Escriba SALIR para cerrar el cliente
-----
> TOKEN200/Registrar entrada de paquete 002
Servidor: ERROR:N3: token inválido
> probando
Servidor: ERROR: formato inválido. Use TOKEN123|Operacion
```

Figura 9: Ingreso de Token Invalido

2.5.5. Caso 5: Caída de N3 y elección de coordinador

Este caso demuestra la tolerancia a fallos. Se apaga el nodo N3 y se espera a que N1 y N2 detecten la ausencia de heartbeats. Luego se ejecuta la elección Bully y el coordinador pasa a ser N2.

```
Run  N1 x N2 x N3 x ClienteTCP x
[ N1] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 106
[ N1] ENVIO -> N2          msg=' Heartbeat              ' C = 108
[ N1] ENVIO -> N3          msg=' Heartbeat              ' C = 109
[ N1] RECEPCION <- N2      msg=' Heartbeat recibido '  marcaMsg= 110
[ N1] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 113
[ N1] ENVIO -> N2          msg=' Heartbeat              ' C = 115
[ N1] ENVIO -> N3          msg=' Heartbeat              ' C = 116
[ N1] RECEPCION <- N2      msg=' Heartbeat recibido '  marcaMsg= 117
[ N1] ENVIO -> N2          msg=' Heartbeat              ' C = 119
[ N1] ENVIO -> N3          msg=' Heartbeat              ' C = 120
[ N1] RECEPCION <- N2      msg=' Heartbeat recibido '  marcaMsg= 121
[ N1] ENVIO -> N2          msg=' Heartbeat              ' C = 123
[ N1] ENVIO -> N3          msg=' Heartbeat              ' C = 124
[ N1] RECEPCION <- N2      msg=' Heartbeat recibido '  marcaMsg= 125
[ N1] ENVIO -> N2          msg=' Heartbeat              ' C = 127
[ N1] ENVIO -> N3          msg=' Heartbeat              ' C = 128
N1 detectó caída de N3 por falta de heartbeat.
Elección Bully en N1: coordinador anterior=N3, nuevo coordinador=N2
[ N1] RECEPCION <- N2      msg=' Heartbeat recibido '  marcaMsg= 129
```

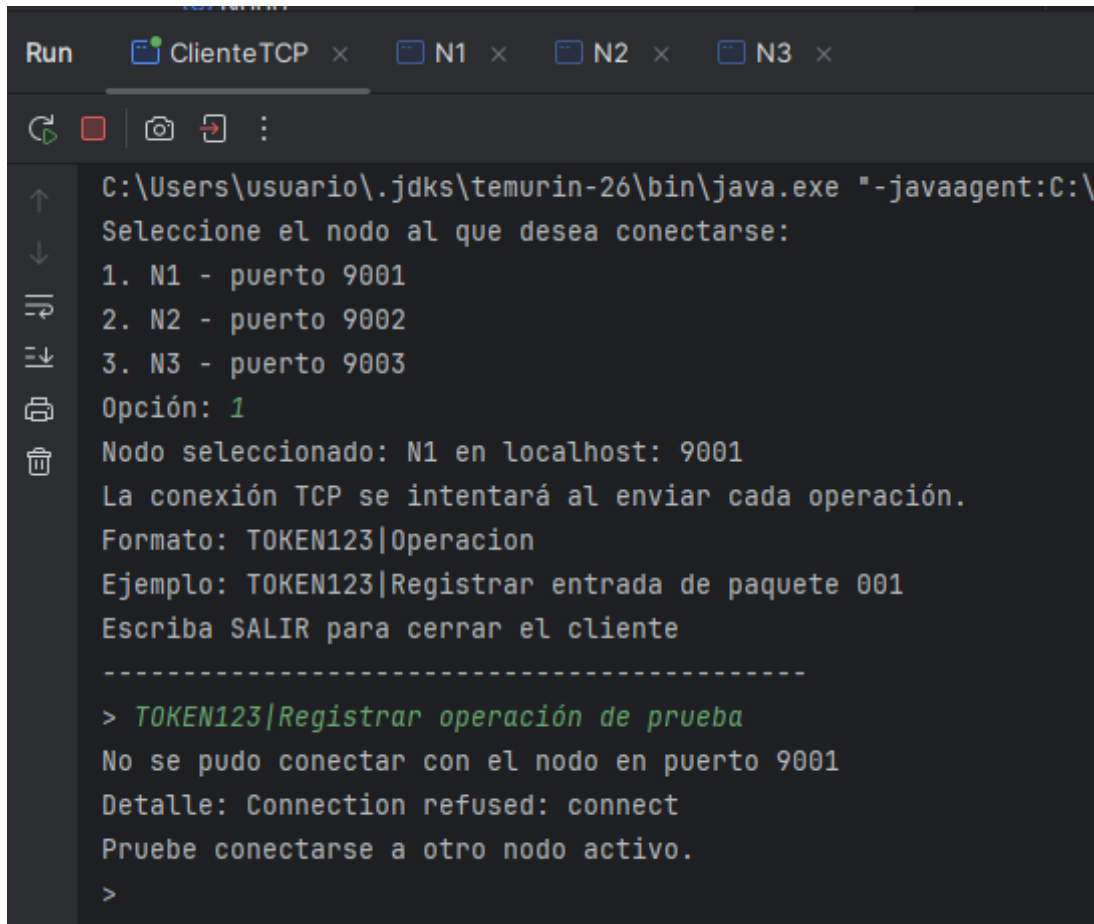
Figura 10: Nodo 1 registrando desactivación de nodo 3 y elección de nuevo coordinador

```
Run N1 x N2 x N3 x ClienteTCP x
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido ' marcaMsg= 107
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido ' marcaMsg= 108
[ N2] ENVIO -> N1         msg=' Heartbeat           ' C = 110
[ N2] ENVIO -> N3         msg=' Heartbeat           ' C = 111
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido ' marcaMsg= 114
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido ' marcaMsg= 115
[ N2] ENVIO -> N1         msg=' Heartbeat           ' C = 117
[ N2] ENVIO -> N3         msg=' Heartbeat           ' C = 118
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido ' marcaMsg= 119
[ N2] ENVIO -> N1         msg=' Heartbeat           ' C = 121
[ N2] ENVIO -> N3         msg=' Heartbeat           ' C = 122
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido ' marcaMsg= 123
[ N2] ENVIO -> N1         msg=' Heartbeat           ' C = 125
[ N2] ENVIO -> N3         msg=' Heartbeat           ' C = 126
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido ' marcaMsg= 127
[ N2] ENVIO -> N1         msg=' Heartbeat           ' C = 129
[ N2] ENVIO -> N3         msg=' Heartbeat           ' C = 130
N2 detectó caída de N3 por falta de heartbeat.
Elección Bully en N2: coordinador anterior=N3, nuevo coordinador=N2
[ N2] RECEPCION <- N1      msg=' Heartbeat recibido ' marcaMsg= 131
```

Figura 11: Nodo 2 registrando desactivación de nodo 3 y elección de nuevo coordinador

2.5.6. Caso 6: Cliente intenta usar un nodo apagado

Este caso demuestra el comportamiento del cliente ante un nodo no disponible. El cliente selecciona un nodo apagado e intenta enviar una operación.

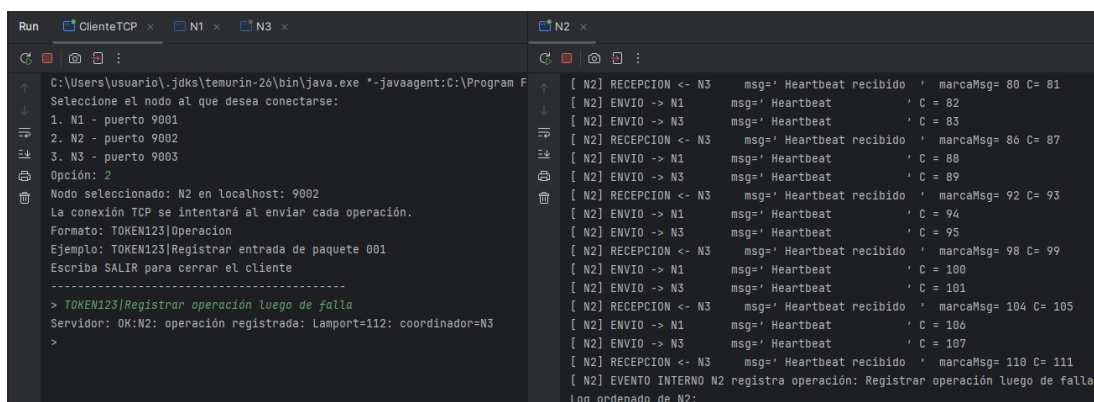


```
Run  ClienteTCP x N1 x N2 x N3 x
C:\Users\usuario\.jdk\temurin-26\bin\java.exe "-javaagent:C:\
Seleccione el nodo al que desea conectarse:
1. N1 - puerto 9001
2. N2 - puerto 9002
3. N3 - puerto 9003
Opción: 1
Nodo seleccionado: N1 en localhost: 9001
La conexión TCP se intentará al enviar cada operación.
Formato: TOKEN123|Operacion
Ejemplo: TOKEN123|Registrar entrada de paquete 001
Escriba SALIR para cerrar el cliente
-----
> TOKEN123|Registrar operación de prueba
No se pudo conectar con el nodo en puerto 9001
Detalle: Connection refused: connect
Pruebe conectarse a otro nodo activo.
>
```

Figura 12: ClienteTCP y su intento fallido de conexión a nodo 3

2.5.7. Caso 7: Cliente se recupera usando otro nodo activo

Este caso demuestra que, aunque un nodo esté apagado, el sistema puede seguir funcionando mediante otro nodo activo. Después de fallar contra un nodo apagado, el cliente se conecta a N1, N2 o N3 según cuál siga funcionando.



```
Run  ClienteTCP x N1 x N2 x
C:\Users\usuario\.jdk\temurin-26\bin\java.exe "-javaagent:C:\Program F
Seleccione el nodo al que desea conectarse:
1. N1 - puerto 9001
2. N2 - puerto 9002
3. N3 - puerto 9003
Opción: 2
Nodo seleccionado: N2 en localhost: 9002
La conexión TCP se intentará al enviar cada operación.
Formato: TOKEN123|Operacion
Ejemplo: TOKEN123|Registrar entrada de paquete 001
Escriba SALIR para cerrar el cliente
-----
> TOKEN123|Registrar operación luego de falla
Servidor: OK:N2: operación registrada: Lamport=112: coordinador=N3
>
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 80 C= 81
[ N2] ENVIO -> N1          msg=' Heartbeat '           ' C = 82
[ N2] ENVIO -> N3          msg=' Heartbeat '           ' C = 83
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 86 C= 87
[ N2] ENVIO -> N1          msg=' Heartbeat '           ' C = 88
[ N2] ENVIO -> N3          msg=' Heartbeat '           ' C = 89
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 92 C= 93
[ N2] ENVIO -> N1          msg=' Heartbeat '           ' C = 94
[ N2] ENVIO -> N3          msg=' Heartbeat '           ' C = 95
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 98 C= 99
[ N2] ENVIO -> N1          msg=' Heartbeat '           ' C = 100
[ N2] ENVIO -> N3          msg=' Heartbeat '           ' C = 101
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 104 C= 105
[ N2] ENVIO -> N1          msg=' Heartbeat '           ' C = 106
[ N2] ENVIO -> N3          msg=' Heartbeat '           ' C = 107
[ N2] RECEPCION <- N3      msg=' Heartbeat recibido '  marcaMsg= 110 C= 111
[ N2] EVENTO INTERNO N2 registra operación: Registrar operación luego de falla
Log ordenado de N2:
```

Figura 13: Registro de operación a nodo 2 activo

```

Run  ClienteTCP x  N1 x  N3 x
C:\Users\usuario\jdk\temurin-20\bin\java.exe -javaagent:C:\Program F
Seleccione el nodo al que desea conectarse:
1. N1 - puerto 9001
2. N2 - puerto 9002
3. N3 - puerto 9003
Opción: 2
Nodo seleccionado: N2 en localhost: 9002
La conexión TCP se intentará al enviar cada operación.
Formato: TOKEN123|Operación
Ejemplo: TOKEN123|Registrar entrada de paquete 001
Escriba SALIR para cerrar el cliente
-----
> TOKEN123|Registrar operación luego de falla
Servidor: OK:N2: operación registrada: Lamport=112: coordinador=N3
>

[ N2] RECEPCION <- N3    msg=' Heartbeat recibido '  marcaMsg= 80 C= 81
[ N2] ENVIO -> N1        msg=' Heartbeat '            ' C = 82
[ N2] ENVIO -> N3        msg=' Heartbeat '            ' C = 83
[ N2] RECEPCION <- N3    msg=' Heartbeat recibido '  marcaMsg= 86 C= 87
[ N2] ENVIO -> N1        msg=' Heartbeat '            ' C = 88
[ N2] ENVIO -> N3        msg=' Heartbeat '            ' C = 89
[ N2] RECEPCION <- N3    msg=' Heartbeat recibido '  marcaMsg= 92 C= 93
[ N2] ENVIO -> N1        msg=' Heartbeat '            ' C = 94
[ N2] ENVIO -> N3        msg=' Heartbeat '            ' C = 95
[ N2] RECEPCION <- N3    msg=' Heartbeat recibido '  marcaMsg= 98 C= 99
[ N2] ENVIO -> N1        msg=' Heartbeat '            ' C = 100
[ N2] ENVIO -> N3        msg=' Heartbeat '            ' C = 101
[ N2] RECEPCION <- N3    msg=' Heartbeat recibido '  marcaMsg= 104 C= 105
[ N2] ENVIO -> N1        msg=' Heartbeat '            ' C = 106
[ N2] ENVIO -> N3        msg=' Heartbeat '            ' C = 107
[ N2] RECEPCION <- N3    msg=' Heartbeat recibido '  marcaMsg= 110 C= 111
[ N2] EVENTO INTERNO N2 registra operación: Registrar operación luego de falla
Log ordenado de N2:

```

Figura 14: Registro de operación a nodo 3 activo

3. Algoritmo Bully

El algoritmo Bully se utiliza para elegir un coordinador entre los nodos activos del sistema. En este prototipo, el coordinador corresponde al nodo activo con el ID más alto. Por esta razón, si los tres nodos están encendidos, el coordinador esperado es N3.

Cuando un nodo deja de responder a los heartbeats, los demás nodos lo marcan como caído. Si el nodo caído era el coordinador, se realiza una nueva elección. En ese caso, el siguiente nodo activo con mayor ID pasa a ser el nuevo coordinador.

Este mecanismo es importante porque permite que el sistema continúe funcionando aunque falle el nodo que actuaba como coordinador.

3.1. Código de NodoInfo.java

La clase `NodoInfo.java` representa la información que un nodo conoce sobre las demás réplicas del sistema. Guarda el ID del nodo, su host, su puerto, la hora del último heartbeat recibido y su estado actual, es decir, si se considera activo o caído.

Esta clase es importante para la detección de fallos, porque permite actualizar el último heartbeat recibido y marcar un nodo como caído cuando deja de responder durante un tiempo determinado.

```

1 package src.ec.edu.uteq.distribuidas.nodo;
2
3 public class NodoInfo {
4
5     private final int id;
6     private final String host;
7     private final int puerto;
8
9     private volatile long ultimoHeartbeat;

```

```

10     private volatile boolean activo;
11
12     public NodoInfo(int id, String host, int puerto) {
13         this.id = id;
14         this.host = host;
15         this.puerto = puerto;
16         this.ultimoHeartbeat = System.currentTimeMillis();
17         this.activo = true;
18     }
19
20     public int getId() {
21         return id;
22     }
23
24     public String getHost() {
25         return host;
26     }
27
28     public int getPuerto() {
29         return puerto;
30     }
31
32     public long getUltimoHeartbeat() {
33         return ultimoHeartbeat;
34     }
35
36     public boolean isActive() {
37         return activo;
38     }
39
40     public void actualizarHeartbeat() {
41         this.ultimoHeartbeat = System.currentTimeMillis();
42         this.activo = true;
43     }
44
45     public void marcarCaido() {
46         this.activo = false;
47     }
48
49     @Override
50     public String toString() {

```

```

51     return "NodoInfo{" +
52         "id=" + id +
53         ", host='" + host + '\',' +
54         ", puerto=" + puerto +
55         ", ultimoHeartbeat=" + ultimoHeartbeat +
56         ", activo=" + activo +
57         '}',
58     }
59 }

```

Listing 5: Código completo de NodoInfo.java

3.2. Código de Bully.java

```

1 package src.ec.edu.uteq.distribuidas.bully;
2
3 import java.util.concurrent.*;
4 import java.util.concurrent.atomic.*;
5
6 public class Bully {
7
8     enum TipoMensaje {ELECTION, OK, COORDINATOR}
9
10    record Mensaje(TipoMensaje tipo, int emisor, int destino) {
11    }
12
13    static class Proceso implements Runnable {
14        final int id;
15        final int totalProcesos;
16        final BlockingQueue<Mensaje>[] buzones;
17        volatile boolean activo;
18        volatile int liderActual = -1;
19        volatile boolean enEleccion = false;
20        static final AtomicInteger liderId = new AtomicInteger(-1);
21
22        @SuppressWarnings(" unchecked ")
23        Proceso(int id, int total, BlockingQueue<Mensaje>[]
24            buzones, boolean activo) {
25            this.id = id;
26            this.totalProcesos = total;
27            this.buzones = buzones;
28            this.activo = activo;

```

```

29     }
30
31     @Override
32     public void run() {
33         Thread.currentThread().setName(" Proceso -" + id);
34         System.out.printf("[P %d] Iniciado . Activo= %b %n", id
35             , activo);
36
37         while (activo) {
38             try {
39                 Mensaje msg = buzones[id].poll(500, TimeUnit.
40                     MILLISECONDS);
41                 if (msg != null) procesarMensaje(msg);
42             } catch (InterruptedException e) {
43                 Thread.currentThread().interrupt();
44                 break;
45             }
46         }
47         System.out.printf("[P %d] Finalizado. %n", id);
48     }
49
50     void procesarMensaje(Mensaje msg) {
51         switch (msg.tipo()) {
52             case ELECTION -> {
53                 System.out.printf("[P %d] Recibe ELECTION deP %
54                     d %n", id, msg.emisor());
55                 // Responder OK al emisor ( soy mas grande )
56                 enviar(TipoMensaje.OK, msg.emisor());
57                 // Iniciar mi propia eleccion si no la tengo
58                 activa
59                 if (!enEleccion) iniciarEleccion();
60             }
61             case OK -> {
62                 System.out.printf("[P %d] Recibe OK de P %d " +
63                     " -> alguien mas grande existe %n", id, msg
64                         .emisor());
65                 enEleccion = false; // otro tomara el control
66             }
67             case COORDINATOR -> {
68                 liderActual = msg.emisor();
69                 liderId.set(liderActual);
70             }
71         }
72     }
73 }

```

```

64         enEleccion = false;
65         System.out.printf("[P %d] NUEVO LIDER : P %d %n",
66             id, liderActual);
67     }
68 }
69
70 void iniciarEleccion() {
71     enEleccion = true;
72     System.out.printf("[P %d] Iniciando ELECTION ... %n",
73         id);
74     boolean hayMasGrande = false;
75     // Enviar ELECTION a todos los procesos con ID mayor
76     for (int i = id + 1; i < totalProcesos; i++) {
77         enviar(TipoMensaje.ELECTION, i);
78         hayMasGrande = true;
79     }
80
81     // Si no hay nadie mas grande , me proclamo lider
82     if (!hayMasGrande) {
83         proclamarLider();
84     } else {
85         // Esperar respuesta OK ( timeout 1.5s)
86         CompletableFuture.delayedExecutor(1500, TimeUnit.
87             MILLISECONDS).execute(() -> {
88             if (enEleccion) {
89                 // Nadie respondio -> soy el mas grande
90                 activo
91                 proclamarLider();
92             }
93         });
94     }
95 }
96
97 void proclamarLider() {
98     liderActual = id;
99     liderId.set(id);
100     enEleccion = false;
101     System.out.printf("[P %d] Me proclamo LIDER ! " + "
102         Enviando COORDINATOR a todos. %n", id);
103     for (int i = 0; i < totalProcesos; i++) {

```

```

100         if (i != id) enviar(TipoMensaje.COORDINATOR, i);
101     }
102 }
103
104 void enviar(TipoMensaje tipo, int destino) {
105     if (destino < totalProcesos) {
106         try {
107             buzones[destino].put(new Mensaje(tipo, id,
108                 destino));
109         } catch (InterruptedException e) {
110             Thread.currentThread().interrupt();
111         }
112     }
113 }
114
115 void simularFallo() {
116     System.out.printf("[P %d] *** SIMULANDO FALLO *** %n",
117         id);
118     activo = false;
119 }
120
121 @SuppressWarnings(" unchecked ")
122 public static void main(String[] args) throws Exception {
123     final int N = 5;
124     BlockingQueue<Mensaje>[] buzones = new BlockingQueue[N
125         ];
126     for (int i = 0; i < N; i++) {
127         buzones[i] = new LinkedBlockingQueue<>();
128     }
129
130     // Crear procesos ( todos activos inicialmente )
131     Proceso[] procs = new Proceso[N];
132     for (int i = 0; i < N; i++) {
133         procs[i] = new Proceso(i, N, buzones, true);
134     }
135
136     ExecutorService exec = Executors.newFixedThreadPool(N);
137     for (Proceso p : procs) exec.submit(p);
138
139     // Inicialmente P4 (id =4, el mayor ) es lider

```

```

140     procs[4].proclamarLider();
141     Thread.sleep(1000);
142
143     System.out.println("\n--- Simulando fallo del lider P4 --
        -\n");
144     procs[4].simularFallo();
145
146     // P1 detecta el fallo e inicia eleccion
147     Thread.sleep(500);
148     procs[1].iniciarEleccion();
149
150     Thread.sleep(3000);
151     System.out.printf(" %nLider final elegido : P %d %n",
152         Proceso.liderId.get());
153     exec.shutdownNow();
154 }
155 }

```

Listing 6: Código completo de Bully.java

3.3. Relación con Nodo.java

Aunque el archivo Bully.java puede contener la lógica relacionada con la elección, la clase Nodo.java es la encargada de aplicar esta coordinación dentro del funcionamiento real del sistema. Esto se debe a que Nodo.java conoce el estado de los demás nodos, recibe heartbeats y detecta cuáles nodos siguen activos.

```

1     private void ejecutarEleccionBully() {
2         int nuevoCoordinador = calcularNodoActivoConMayorId();
3
4         if (nuevoCoordinador != coordinadorId) {
5             System.out.println(
6                 "Elecci n Bully en N" + id +
7                 ": coordinador anterior=N" +
8                 coordinadorId +
9                 ", nuevo coordinador=N" +
10                nuevoCoordinador
11            );
12
13            coordinadorId = nuevoCoordinador;
14        } else {
15            System.out.println(

```



```

14         "Elecci n Bully en N" + id +
15         ": se mantiene coordinador=N" +
            coordinadorId
16     );
17 }
18 }

```

Listing 7: Fragmento relacionado con elecci3n Bully en Nodo.java

4. Evento

La clase `Evento.java` representa las operaciones registradas dentro del sistema distribuido. Cada evento contiene una marca de tiempo l3gica de Lamport, el identificador del nodo donde se origin3 y una descripci3n de la operaci3n realizada.

Esta clase es importante porque permite guardar los eventos de forma ordenada. El ordenamiento se realiza primero por la marca de tiempo Lamport y luego por el ID del nodo como criterio de desempate. Esto ayuda a mantener un registro l3gico de los eventos ocurridos en el sistema, incluso cuando estos provienen de diferentes nodos.

4.1. C3digo de Evento.java

```

1 package src.ec.edu.uteq.distribuidas.evento;
2
3 public class Evento implements Comparable<Evento> {
4
5     private final int timestampLamport;
6     private final int nodoId;
7     private final String descripcion;
8
9     public Evento(int timestampLamport, int nodoId, String
        descripcion) {
10         this.timestampLamport = timestampLamport;
11         this.nodoId = nodoId;
12         this.descripcion = descripcion;
13     }
14
15     public int getTimestampLamport() {
16         return timestampLamport;
17     }
18
19     public int getNodoId() {

```

```

20         return nodoId;
21     }
22
23     public String getDescripcion() {
24         return descripcion;
25     }
26
27     @Override
28     public int compareTo(Evento otro) {
29         int comparacionTiempo = Integer.compare(this.
30             timestampLamport, otro.timestampLamport);
31
32         if (comparacionTiempo != 0) {
33             return comparacionTiempo;
34         }
35
36         return Integer.compare(this.nodoId, otro.nodoId);
37     }
38
39     @Override
40     public String toString() {
41         return "[L=" + timestampLamport + ", Nodo=" + nodoId + "] "
42             + descripcion;
43     }
44 }

```

Listing 8: Código completo de Evento.java

5. Lamport

Los relojes lógicos de Lamport permiten ordenar eventos en un sistema distribuido sin depender de un reloj físico global. En este prototipo, cada nodo posee su propio reloj lógico. El reloj aumenta cuando ocurre un evento interno, cuando se envía un mensaje y cuando se recibe un mensaje desde otro nodo.

Este mecanismo es importante porque en un sistema distribuido no se puede asumir que todos los nodos tienen exactamente la misma hora física. Por eso, Lamport permite establecer una relación lógica entre eventos y mensajes.

5.1. Código de LamportClock.java

```

1 package src.ec.edu.uteq.distribuidas.lamport;

```

```

2
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class LamportClock {
6
7     private final AtomicInteger contador = new AtomicInteger
8         (0);
9     private final String nombreProceso;
10
11     public LamportClock(String nombre) {
12         this.nombreProceso = nombre;
13     }
14
15     public int eventoInterno(String descripcion) {
16         int t = contador.incrementAndGet();
17         System.out.printf("[ %s] EVENTO INTERNO %-15s C= %d %n",
18             nombreProceso, descripcion, t);
19         return t;
20     }
21
22     public int enviar(String destino, String mensaje) {
23         int t = contador.incrementAndGet();
24         System.out.printf("[ %s] ENVIO -> %-8s msg= %-20 s C =
25             %d %n", nombreProceso, destino, mensaje, t);
26         return t; // esta marca viaja con el mensaje
27     }
28
29     public int recibir(String origen, int marcaMensaje,
30         String mensaje) {
31         int nuevo;
32         while (true) {
33             int actual = contador.get();
34             nuevo = Math.max(actual, marcaMensaje) + 1;
35             if (contador.compareAndSet(actual, nuevo)) break
36                 ;
37         }
38         System.out.printf("[ %s] RECEPCION <- %-6s msg= %-20 s
39             " + " marcaMsg= %d C= %d %n", nombreProceso, origen,
40             mensaje, marcaMensaje, nuevo);
41         return nuevo;
42     }
43 }

```

```

39
40     public int getContador() {
41         return contador.get();
42     }
43
44     public String getNombre() {
45         return nombreProceso;
46     }
47 }

```

Listing 9: Código completo de LamportClock.java

5.2. Código de Lamport.java

```

1  package src.ec.edu.uteq.distribuidas.lamport;
2
3  import java.util.concurrent.BlockingQueue;
4  import java.util.concurrent.CountDownLatch;
5  import java.util.concurrent.ExecutionException;
6  import java.util.concurrent.ExecutorService;
7  import java.util.concurrent.Executors;
8  import java.util.concurrent.Future;
9  import java.util.concurrent.LinkedBlockingQueue;
10
11 public class Lamport {
12
13     public static void main(String[] args) throws
14         InterruptedException, ExecutionException {
15
16         // Canales de comunicaci n. BlockingQueue simula el paso
17         de mensajes.
18         BlockingQueue<int[]> p1aP2 = new LinkedBlockingQueue<>();
19         BlockingQueue<int[]> p2aP3 = new LinkedBlockingQueue<>();
20         BlockingQueue<int[]> p3aP1 = new LinkedBlockingQueue<>();
21
22         ExecutorService exec = Executors.newFixedThreadPool(3);
23         CountDownLatch inicio = new CountDownLatch(3);
24
25         System.out.println("== Simulaci n de Relojes de Lamport
26             ==");
27         System.out.println("Formato: [Proceso] TIPO ... C=valor\n");
28         ;

```

```

25
26 // Proceso P1
27 Future<?> f1 = exec.submit(() -> {
28     LamportClock c = new LamportClock("P1");
29
30     inicio.countDown();
31
32     try {
33         inicio.await();
34
35         c.eventoInterno("a");
36
37         int t = c.enviar("P2", "Hola P2");
38         p1aP2.put(new int[]{t});
39
40         c.eventoInterno("e");
41
42         int[] marcaP3 = p3aP1.take();
43         c.recibir("P3", marcaP3[0], "Respuesta de P3");
44
45         c.eventoInterno("f");
46
47     } catch (InterruptedException e) {
48         Thread.currentThread().interrupt();
49     }
50 });
51
52 // Proceso P2
53 Future<?> f2 = exec.submit(() -> {
54     LamportClock c = new LamportClock("P2");
55
56     inicio.countDown();
57
58     try {
59         inicio.await();
60
61         c.eventoInterno("c");
62
63         int[] marcaP1 = p1aP2.take();
64         c.recibir("P1", marcaP1[0], "Hola P2");
65

```

```

66         c.eventoInterno("g");
67
68         int t = c.enviar("P3", "Hola P3");
69         p2aP3.put(new int[]{t});
70
71         c.eventoInterno("h");
72
73     } catch (InterruptedException e) {
74         Thread.currentThread().interrupt();
75     }
76 });
77
78 // Proceso P3
79 Future<?> f3 = exec.submit(() -> {
80     LamportClock c = new LamportClock("P3");
81
82     inicio.countDown();
83
84     try {
85         inicio.await();
86
87         c.eventoInterno("i");
88
89         int[] marcaP2 = p2aP3.take();
90         c.recibir("P2", marcaP2[0], "Hola P3");
91
92         c.eventoInterno("k");
93
94         int t = c.enviar("P1", "Hola P1");
95         p3aP1.put(new int[]{t});
96
97     } catch (InterruptedException e) {
98         Thread.currentThread().interrupt();
99     }
100 });
101
102 f1.get();
103 f2.get();
104 f3.get();
105
106 exec.shutdown();

```

```
107
108         System.out.println("\n=== Simulaci n completada ===");
109     }
110 }
```

Listing 10: Código completo de Lamport.java

5.3. Importancia dentro del prototipo

En el prototipo, Lamport se usa para registrar operaciones y mensajes internos con una marca lógica. Esto permite observar cómo avanza el tiempo lógico del sistema conforme ocurren operaciones del cliente y heartbeats entre nodos.

Los heartbeats también incrementan el reloj Lamport, ya que son mensajes distribuidos reales entre los nodos. Por esa razón, es normal que las marcas Lamport crezcan rápidamente durante la ejecución.

Cuestionario del SGA

PREGUNTA

16

Finalizado

1.00 / 1.00

Un servicio se traslada de un servidor a otro durante la madrugada, cuando no hay clientes conectados; al reanudar la operación, los clientes lo siguen invocando con el mismo nombre lógico sin percibir cambio alguno. ¿Qué transparencia se evidencia principalmente?

Seleccione una alternativa:

- ☐ a. Transparencia de ubicación.
- ☐ b. Transparencia de acceso.
- ☒ c. Transparencia de migración.
- ☐ d. Transparencia de reubicación.

La respuesta correcta es: Transparencia de migración.

PREGUNTA

17

Finalizado

1.00 / 1.00

Un acuerdo de nivel de servicio (SLA) promete una disponibilidad anual del 99.9 %. ¿Aproximadamente cuánto tiempo de inactividad permite al año, y qué implicaría elevarlo al 99.99 %?

Seleccione una alternativa:

- ☒ a. = 8.76 horas al año; al 99.99 % se reduciría a = 52.6 minutos al año.
- ☐ b. = 88 horas al año; el 99.99 % lo empeoraría.
- ☐ c. = 1 minuto al año; el 99.99 % lo duplicaría.
- ☐ d. = 3.65 días al año; el 99.99 % no cambiaría nada.

La respuesta correcta es: = 8.76 horas al año; al 99.99 % se reduciría a = 52.6 minutos al año.

PREGUNTA

18

Finalizado

1.00 / 1.00

Un carrito de compras prioriza la disponibilidad (AP) y emplea consistencia eventual. Un usuario agrega un ítem y, al recargar de inmediato, no lo ve; segundos después sí aparece. ¿Cómo se explica esto correctamente?

Seleccione una alternativa:

- ☒ a. Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.
- ☐ b. El sistema cambió automáticamente a un modo CP e indica que el sistema no es tolerante a particiones.
- ☐ c. Es un fallo del sistema que viola el teorema CAP.
- ☐ d. Es el comportamiento esperado: alta consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata devuelve datos actualizados.

La respuesta correcta es: Es el comportamiento esperado: bajo consistencia eventual las réplicas convergen con el tiempo, por lo que una lectura inmediata puede devolver datos obsoletos.

PREGUNTA

19

Finalizado

0.00 / 1.00

Relacione: Situación de seguridad y propiedad que se protege

Relacione la opción correcta:

Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor.

Integridad



Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido.

Autorización



El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito.

Confidencialidad



Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol.

Autenticación



La respuesta correcta es: Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido. → No repudio. El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito. → Integridad. Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol. → Autorización. Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor. → Autenticación

PREGUNTA

20

Finalizado

0.00 / 1.00

Un equipo intenta mejorar el rendimiento de una aplicación que envía miles de mensajes muy pequeños entre dos nodos. Duplica el ancho de banda del enlace, pero el rendimiento prácticamente no cambia. ¿Qué falacia de la computación distribuida explica este resultado?

Seleccione una alternativa:

- ☐ a. "El ancho de banda es infinito".
- ☒ b. "La red es confiable" ✖
- ☐ c. "La latencia es cero".
- ☐ d. "El transporte es gratuito".

La respuesta correcta es: "La latencia es cero".